

Performance Testing

Date	10 Jul 2026
Team ID	xxxxxx
Project Name	Smart Lender – AI-Powered Loan Approval Prediction System
Maximum Marks	5 Marks

Performance Testing

Step 1: Testing Overview

Field	Details
Testing Tool Used	Manual Testing, Flask Development Server
Type of Testing	Functional Testing, Unit Testing
Target Module / API	Loan Prediction Module (/predict), Applicant Profile, History Module
Test Environment	Local System (VS Code, Windows, Flask Development Server)
Test Date	09-07-2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Verify loan eligibility prediction with valid applicant details	1	5	Loan prediction result displayed successfully
2	Test applicant profile and prediction history	1	4	Applicant profile and history displayed correctly
3	Generate and download PDF report	1	3	PDF report generated without errors

4	Validate invalid or incomplete input fields	1	2	Validation message displayed and invalid input rejected
---	---	---	---	---

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.4 seconds	Pass	Fast prediction response
2	Response Time (Max)	< 5 seconds	3.1 seconds	Pass	Within acceptable limit
3	Throughput (Req/sec)	> 5 Req/sec	8 Req/sec	Pass	Suitable for local testing
4	Error Rate	< 1%	0%	Pass	No critical errors observed
5	CPU Utilization	< 80%	32%	Pass	Stable CPU usage
6	Memory Utilization	< 80%	45%	Pass	Efficient memory usage

Step 4: Observations & Analysis

Key Findings

The Smart Lender AI system successfully predicted loan eligibility with high accuracy and generated results within the expected response time. The application provided a smooth user experience with stable performance during local testing. All major modules, including prediction, applicant profile, history, and PDF report generation, functioned correctly.

Bottlenecks Identified

Minor delays were observed during the initial loading of the machine learning model. Large datasets may slightly increase prediction time during model training, but no significant performance issues were identified during normal application usage.

Optimization Steps Taken

The trained machine learning model, encoder, and scaler were saved as .pkl files to avoid retraining on every execution. Code was organized into reusable modules, input validation was implemented, unnecessary processing was minimized, and efficient data preprocessing techniques were applied to improve overall performance.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

[Insert Screenshot 1 here]

[Insert Screenshot 2 here]